

Polishing <chrono>

Contents

- [Introduction](#)
- [Implementation](#)
- [Wording](#)

Introduction

This paper proposes a few minor additions to the chrono library to make it easier to use and more uniform:

- Increment and decrement operators for `time_point`.
- Alternative rounding modes for durations and `time_points`:
 - `floor`: round towards negative infinity.
 - `ceil`: round towards positive infinity.
 - `round`: round towards nearest, to even on tie.
- `abs` *only* for signed duration types.

These are *minor* additions that make chrono easier to use. They have been implemented **and** used for years. They should have been part of the original chrono proposal for C++11, but simply slipped through the cracks by no one's fault but my own.

The rounding modes differ from `duration_cast` and `time_point_cast` in that these existing rounding functions truncate towards zero: downwards for positive values and upwards for negative values.

Implementation

The implementation is shown below for the utility functions. This presentation is meant to:

- Fully inform the intent of this proposal.
- Immediately convey the scope and the small-ness of this proposal.
- Show the ease of implementation of this proposal.

This implementation is consistent with that which has been up on [my personal website](#) for years.

```
namespace std { namespace chrono {

namespace detail
{

template <class T>
struct is_duration
    : public false_type
    {};

template <class Rep, class Period>
struct is_duration<duration<Rep, Period>>
    : public true_type
    {};

} // namespace detail

template <class To, class Rep, class Period,
          class = enable_if_t<detail::is_duration<To>{}>>
constexpr
To
```

```

floor(const duration<Rep, Period>& d)
{
    To t = duration_cast<To>(d);
    if (t > d)
        --t;
    return t;
}

template <class To, class Rep, class Period,
          class = enable_if_t<detail::is_duration<To>{}>>
constexpr
To
ceil(const duration<Rep, Period>& d)
{
    To t = duration_cast<To>(d);
    if (t < d)
        ++t;
    return t;
}

template <class To, class Rep, class Period,
          class = enable_if_t<detail::is_duration<To>{}
            && !treat_as_floating_point<typename To::rep>{}>>
constexpr
To
round(const duration<Rep, Period>& d)
{
    To t0 = floor<To>(d);
    To t1 = t0 + To{1};
    auto diff0 = d - t0;
    auto diff1 = t1 - d;
    if (diff0 == diff1)
    {
        if (t0.count() & 1)
            return t1;
        return t0;
    }
    else if (diff0 < diff1)
        return t0;
    return t1;
}

template <class To, class Clock, class FromDuration,
          class = enable_if_t<detail::is_duration<To>{}>>
constexpr
time_point<Clock, To>
floor(const time_point<Clock, FromDuration>& tp)
{
    return time_point<Clock, To>{floor<To>(tp.time_since_epoch())};
}

template <class To, class Clock, class FromDuration,
          class = enable_if_t<detail::is_duration<To>{}>>
constexpr
time_point<Clock, To>
ceil(const time_point<Clock, FromDuration>& tp)
{
    return time_point<Clock, To>{ceil<To>(tp.time_since_epoch())};
}

template <class To, class Clock, class FromDuration,
          class = enable_if_t<detail::is_duration<To>{}
            && !treat_as_floating_point<typename To::rep>{}>>
constexpr
time_point<Clock, To>
round(const time_point<Clock, FromDuration>& tp)
{
    return time_point<Clock, To>{round<To>(tp.time_since_epoch())};
}

template <class Rep, class Period,
          class = enable_if_t
            <
                duration<Rep, Period>::min() < duration<Rep, Period>::zero()
            > >
constexpr
duration<Rep, Period>
abs(duration<Rep, Period> d)
{

```

```

    return d >= d.zero() ? d : -d;
}

} } // namespace std::chrono

```

Wording

- Add to 20.12.2 Header <chrono> synopsis [time.syn], under duration_cast:

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration floor(const duration<Rep, Period>& d);

template <class ToDuration, class Rep, class Period>
constexpr ToDuration ceil(const duration<Rep, Period>& d);

template <class ToDuration, class Rep, class Period>
constexpr ToDuration round(const duration<Rep, Period>& d);

```

- Add to 20.12.2 Header <chrono> synopsis [time.syn], under time_point_cast:

```

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
floor(const time_point<Clock, Duration>& tp);

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
ceil(const time_point<Clock, Duration>& tp);

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
round(const time_point<Clock, Duration>& tp);

```

- Add a new section to 20.12.2 Header <chrono> synopsis [time.syn] (*specialized algorithms*: — within namespace std::chrono):

```

template <class Rep, class Period>
constexpr duration<Rep, Period> abs(duration<Rep, Period> d)

```

- Add to 20.12.5.7 <duration_cast> [time.duration.cast]:

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration floor(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is an instantiation of duration.

Returns: The largest result t representable in ToDuration that can be returned from duration_cast<ToDuration>(d) for which t <= d.

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration ceil(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is an instantiation of duration.

Returns: The smallest result t representable in ToDuration that can be returned from duration_cast<ToDuration>(d) for which t >= d.

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration round(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is an instantiation of duration, and treat_as_floating_point<typename ToDuration::rep>::value is false.

Returns: The result t representable in ToDuration which is closest in value to d. If d falls exactly half way between t and an adjacent representable value in ToDuration, then the value of ToDuration which is even is returned.

- Add a new section to 20.12.5 [time.duration]: duration algorithms [time.duration.alg].

```

template <class Rep, class Period>
constexpr duration<Rep, Period> abs(duration<Rep, Period> d)

```

Remarks: This function shall not participate in overload resolution unless duration<Rep, Period>::min() < duration<Rep, Period>::zero().

Returns: If `d >= d.zero()`, return `d`, otherwise return `-d`.

- Add to 20.12.6 Class template `time_point` [`time.point`], in the synopsis, in the section *arithmetic*:

```
time_point& operator++();  
time_point operator++(int);  
time_point& operator--();  
time_point operator--(int);
```

- Add to 20.12.6.3 `time_point` arithmetic [`time.point.arithmetic`]:

```
time_point& operator++();
```

Effects: `++d_`.

Returns: `*this`.

```
time_point operator++(int);
```

Returns: `time_point(d_++)`.

```
time_point& operator--();
```

Effects: `--d_`.

Returns: `*this`.

```
time_point operator--(int);
```

Returns: `time_point(d_--)`.

- Add to 20.12.6.7 `time_point_cast` [`time.point.cast`]:

```
template <class ToDuration, class Clock, class Duration>  
constexpr time_point<Clock, ToDuration>  
floor(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is an instantiation of duration.

Returns: `time_point<Clock, ToDuration>{floor<ToDuration>(tp.time_since_epoch())}`.

```
template <class ToDuration, class Clock, class Duration>  
constexpr time_point<Clock, ToDuration>  
ceil(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is an instantiation of duration.

Returns: `time_point<Clock, ToDuration>{ceil<ToDuration>(tp.time_since_epoch())}`.

```
template <class ToDuration, class Clock, class Duration>  
constexpr time_point<Clock, ToDuration>  
round(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is an instantiation of duration, and `treat_as_floating_point<typename ToDuration::rep>::value` is false.

Returns: `time_point<Clock, ToDuration>{round<ToDuration>(tp.time_since_epoch())}`.